سؤالات نظری (۱۰۰ نمره)

سؤال ۱ **[۲۵ نمره]**

تحلیل و حدس ۸ الگوریتم و ۳ مسئله جستجوی محلی

بخش اول: مسئله ۱۰۰ وزیر
مسئله چیدمان **۱۰۰ وزیر** (100-Queens, $N = 100$) را در نظر بگیرید:

- **نمایش حالت:** یک آرایه به طول ۱۰۰ که در آن اندیس i نشان‌دهنده سطر و مقدار $\text{state}[i]$ ستون قرارگیری وزیر در آن سطر است ($0 \leq \text{state}[i] < 100$). با این نمایش، در هر سطر همواره دقیقاً یک وزیر مستقر است.
- **تابع هزینه (هدف):** تعداد کل جفت‌وزیرهای تهدیدکننده (برخوردهای ستونی و برخوردهای قطری). حالت هدف مطلوب دارای دقیقاً **۰ برخورد** است. بیشینه نظری تعداد برخوردها برابر با $\binom{100}{2} = 4950$ است.
- **فضای همسایگی:** جابجایی یک وزیر در سطر i به ستون دلخواه دیگر $j \neq \text{state}[i]$ (اندازه فضای همسایگی در هر گام برابر با $100 \times 99 = 9900$ حرکت ممکن است).

هشت الگوریتم جستجوی محلی بر روی این مسئله اجرا شده و رفتار تغییرات هزینه آن‌ها در طول ۱۵۰ تکرار ثبت شده است. منحنی‌های حاصل با شناسه‌های ناشناس Algorithm A تا Algorithm H مشخص شده‌اند.

فهرست ۸ الگوریتم اجرا شده و ابرپارامترهای آن‌ها:

۱. Steepest-Ascent Hill-Climbing (اجرای اول): ارزیابی تمام همسایه‌ها در هر گام و انتخاب بهترین حرکت (بیشترین کاهش هزینه).

۲. Steepest-Ascent Hill-Climbing (اجرای دوم): اجرای مجدد همان الگوریتم از یک وضعیت آغازین تصادفی دیگر.

۳. Genetic Algorithm with Elitism: اندازه جمعیت $\text{Pop} = 50$، حفظ ۲ فرد برتر به عنوان نخبه ($E = 2$)، عملگر تقاطع 2-point crossover با احتمال $p_c = 0.85$ و جهش تصادفی (mutation) با $p_m = 0.08$. در نمودار، هزینه بهترین فرد هر نسل رسم شده است.

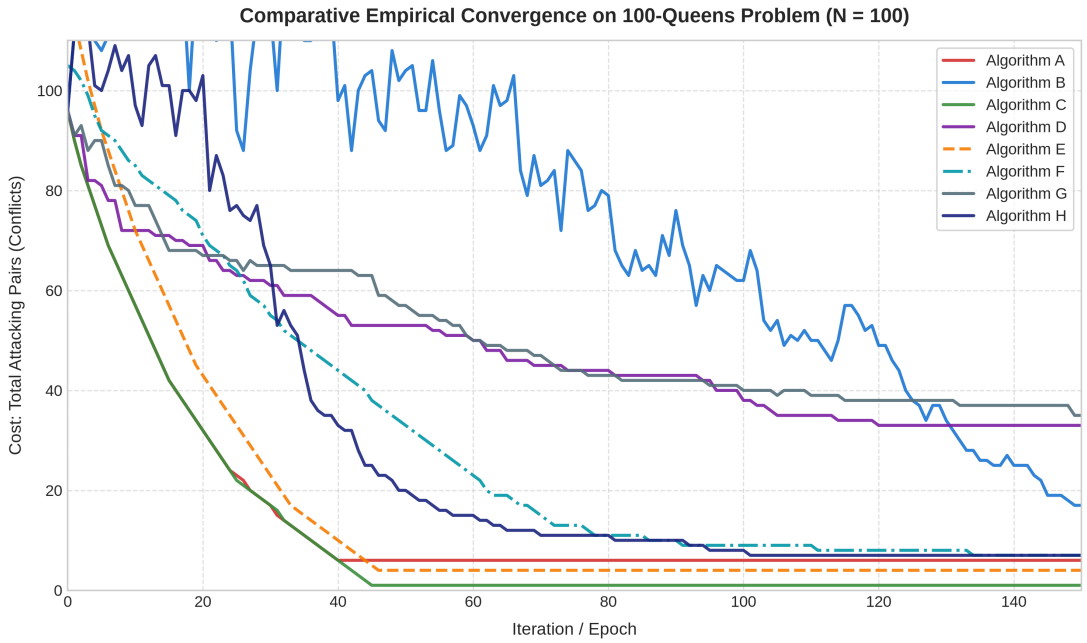
۴. Genetic Algorithm without Elitism: اندازه جمعیت $\text{Pop} = 50$، بدون حفظ نخبه ($E = 0$)، تقاطع با $p_c = 0.85$ و جهش با $p_m = 0.08$.

۵. Simulated Annealing - Slow Cooling: دمای اولیه $T_0 = 35.0$، نرخ سرمایش آرام با $T_{k+1} = 0.965 \cdot T_k$ و قاعده پذیرش Metropolis با $P = \exp(-\Delta E / T)$.

۶. Simulated Annealing - Fast Cooling / Quenching: دمای اولیه $T_0 = 35.0$، نرخ سرمایش سریع با $T_{k+1} = 0.88 \cdot T_k$ و همان قاعده پذیرش Metropolis.

۷. Local Beam Search - Steepest-First: پهنای پرتو $K = 5$. در هر گام، تمام فرزندان تولیدشده توسط ۵ وضعیت پرتو تجمیع شده و ۵ وضعیت برتر با کمترین هزینه انتخاب می‌شوند.

۸. Local Beam Search - Random Better Pick: پهنای پرتو $K = 5$. برای هر وضعیت پرتو، همسایه‌ها به صورت تصادفی بررسی می‌شوند تا اولین حرکت بهبوددهنده ($\Delta E < 0$) پیدا شود و پرتو به آن منتقل شود.



الف) تطبیق الگوریتم‌ها (Identification): مشخص کنید هر یک از شناسه‌های Algorithm A تا Algorithm H مربوط به کدام الگوریتم از میان ۸ الگوریتم بالاست. دلیل خود را به طور کامل تحلیل کنید.

ب) تحلیل نمودارها (Curve Analysis): رفتار هر منحنی را در طول تکرارها تحلیل و توجیه نمایید.

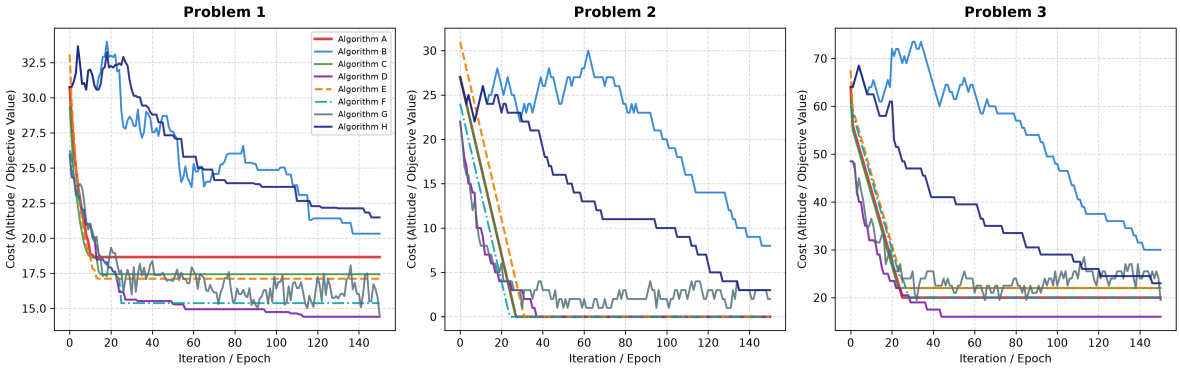
ج) مقایسه منحنی‌ها:

- دو منحنی مربوط به Hill-Climbing را با هم مقایسه کرده و دلیل تفاوت رفتار آن‌ها را بنویسید.
- تأثیر نرخ سرمایش را در دو منحنی تبرید شبیه‌سازی‌شده (Simulated Annealing) تحلیل کنید.
- تأثیر نخبه‌گرایی (Elitism) را در دو منحنی الگوریتم ژنتیک بررسی کنید.
- دو حالت الگوریتم Beam Search را با یکدیگر مقایسه نمایید.

بخش دوم: آزمون روی ۳ مسئله ناشناخته

همان ۸ الگوریتم محلی (Algorithm A تا Algorithm H) بر روی **۳ مسئله بهینه‌سازی ناشناخته** با ابعاد یکسان ($N = ۶۰$) و فضای همسایگی ۱-همینگ اجرا شده‌اند. کمینه مطلوب در هر سه مسئله هزینه صفر ($h = ۰$) است. سه ساختار فضای جستجو به شرح زیر تعریف شده‌اند:

- **فضای Smooth Crater:** یک فضای محدب و هموار کاسه‌ای شکل که در آن هیچ بهینه محلی یا بن‌بستی وجود ندارد و هر حرکت رو به پایین مستقیماً به مرکز ($h = ۰$) هدایت می‌شود.
- **فضای Rugged Badlands:** فضایی با ناهمواری بسیار شدید، شامل هزاران بهینه محلی کم‌عمق و عمیق، همراه با برهم‌کنش قوی بین متغیرها (epistasis، $N = ۶۰, K = ۵$).
- **فضای Mirage Trap:** یک فضای فریبنده (deceptive) که گرادیان موضعی آن الگوریتم‌ها را به سمت یک بهینه محلی جاذب در $h \approx ۲۰$ هدایت می‌کند، در حالی که کمینه سراسری ($h = ۰$) پشت یک دیواره صخره‌ای قرار دارد.



د) مشخص کنید هر یک از سه نمودار (Problem 1 تا Problem 3) متناظر با کدام‌یک از سه فضای فوق است. دلیل تطبیق خود را بر اساس رفتار و همگرایی الگوریتم‌ها شرح دهید.

هـ) رفتار و علل ناکامی یا موفقیت الگوریتم‌های مختلف را در این سه فضا تحلیل و مقایسه نمایید.

سؤال ۲ **[۷ نمره]**

گراف جهت‌دار با مجموعه رأس‌های $V = \{A, B, C, D, E, F\}$ را بدون طوقه (self-loop) در نظر بگیرید. هزینه‌ها توسط تابع h تعیین شده و مقادیر کمتر برتر هستند (lower is better). فرض کنید مقادیر هزینه برای همه رأس‌ها متمایزند. هر گره u فقط همسایه‌هایی نظیر v را بررسی می‌کند که در آن $(u, v) \in E$ باشد.
الگوریتم‌های زیر با تعاریف استاندارد عمل می‌کنند:

- **Steepest-Ascent:** به همسایه‌ای با کمترین هزینه می‌رود مشروط بر اینکه اکیداً بهتر باشد؛ در غیر این صورت متوقف می‌شود.
- **First-Choice:** همسایه‌ها را به ترتیب الفبایی (alphabetical order) بررسی کرده و به اولین همسایه‌ای که اکیداً بهتر باشد منتقل می‌شود؛ در غیر این صورت متوقف می‌گردد.
- **Beam Search ($K = ۲$):** تمام فرزندان وضعیت‌های پرتو را در یک استخر تجمیع کرده، پس از حذف وضعیت‌های تکراری، ۲ گره با کمترین هزینه را نگه می‌دارد.

الف) ساختار اجباری و شمارش: فرض کنید گراف G دارای **دقیقاً ۵ یال** است ($|E| = ۵$) و h یک جایگشت از مجموعه $\{۱, ۲, ۳, ۴, ۵, ۶\}$ است. اگر هر سه شرط زیر به طور همزمان برقرار باشند:

۱. Steepest-ascent از مبدأ A مسیر $A \to D \to F$ را طی کرده و متوقف شود.

۲. First-choice از مبدأ A مسیر $A \to B \to C \to E$ را طی کرده و متوقف شود.

۳. Beam search با $K = ۲$ از پرتو آغازین $S_۰ = \{A, B\}$ به ترتیب به $S_۱ = \{C, D\}$ و سپس $S_۲ = \{E, F\}$ برسد.

- مجموعه یال‌های گراف (E) را به صورت دقیق تعیین کنید.
- ثابت کنید مقادیر $h(A)$ و $h(B)$ یکتا هستند و مقدار آن‌ها را بیابید.
- تعداد کل انتساب‌های معتبر برای تابع هزینه h را شمارش کرده و استدلال ریاضی خود را بنویسید.

سؤال ۳ **[۱۵ نمره]**

در این سؤال روابط مفهومی و قابلیت بازتولید الگوریتم‌های جستجوی محلی را بررسی می‌کنیم.

الف) شبیه‌سازی تپه‌نوردی تصادفی با الگوریتم ژنتیک (Simulating Stochastic HC via GA): چگونه می‌توان مؤلفه‌ها و ابرپارامترهای یک الگوریتم ژنتیک (شامل اندازه جمعیت N، احتمال و عملگر تقاطع، احتمال و نحوه جهش، سازوکار انتخاب والدین و نخبه‌گرایی) را طوری تنظیم کرد که رفتار الگوریتم دقیقاً معادل با جستجوی تپه‌نوردی تصادفی تک‌مسیره (تولید یک همسایه تصادفی در هر گام و حرکت به آن در صورت بهبود یا برابری) گردد؟ تنظیم دقیق هر مؤلفه را شرح دهید.

ب) شبیه‌سازی تبرید شبیه‌سازی‌شده با الگوریتم ژنتیک (Simulating SA via GA): آیا می‌توان الگوریتم Simulated Annealing را نیز با بازتنظیم سازوکارهای الگوریتم ژنتیک شبیه‌سازی کرد؟ به موارد زیر پاسخ دهید:

- اندازه جمعیت (N)، احتمال تقاطع (p_c) و نحوه اعمال جهش چه مقادیری باید داشته باشند تا متناظر با تولید همسایه تصادفی در SA گردند؟
- چه تغییری در سازوکار جایگزینی و پذیرش فرزند برای پیاده‌سازی قاعده پذیرش متروپولیس (Metropolis) لازم است؟
- نقش شماره نسل‌ها در الگوریتم ژنتیک متناظر با کدام مؤلفه در جدول زمانبندی سرمایش (Cooling Schedule) در SA خواهد بود؟

ج) تحلیل ادعا درباره رفتار حدی تبرید: یک دانشجو ادعا می‌کند: «الگوریتم Simulated Annealing هنگامی که دما به سمت صفر میل کند ($T \to ۰$) و در هر تکرار یک همسایه تصادفی تولید شود، رفتاری کاملاً هم‌ارز با الگوریتم تپه‌نوردی اولین انتخاب (First-Choice Hill Climbing) خواهد داشت.» آیا این ادعا درست است؟ دلیل بیاورید.

سؤال ۴ [۱۸ نمره]

کروموزوم‌هایی به فرم $X = abcd$ با طول ثابت ۴ رقم ده‌دهی را در نظر بگیرید ($a, b, c, d \in \{۰, ۱, \dots, ۹\}$). اندازه جمعیت اولیه برابر با ۴ بوده و شامل کروموزوم‌های زیر است:

$$X_۱ = ۲۷۴۶, \quad X_۲ = ۸۳۵۱, \quad X_۳ = ۴۹۲۸, \quad X_۴ = ۶۱۷۳$$

تابع برازش (Fitness function) برای هر کروموزوم به صورت زیر تعریف شده است:

$$\text{Fitness}(X) = ۴۰ - \Big(|a - ۵| + |b - ۲| + |c - ۸| + |d - ۴| \Big)$$

قواعد حاکم بر چرخه تکاملی در هر نسل به شرح زیر است:

۱. **ارزیابی برازش:** مقدار برازش تمام کروموزوم‌های جمعیت جاری محاسبه می‌شود.

۲. **انتخاب و تقاطع (Crossover):** دو کروموزوم با بیشترین مقدار برازش انتخاب شده و با روش تقاطع یک‌نقطه‌ای (One-point Crossover) از نقطه میانی ($ab \mid cd$)، دو فرزند جدید ($C_۱$ و $C_۲$) تولید می‌کنند.

۳. **تشکیل جمعیت موقت:** پس از تقاطع، جمعیت موقت شامل ۶ کروموزوم خواهد بود (۴ کروموزوم نسل قبل + ۲ فرزند تازه تولیدشده).

۴. **جهش (Mutation):** فرزندی که مقدار برازش کمتری دارد برای جهش انتخاب می‌شود (در صورت تساوی، یکی به دلخواه برگزیده می‌شود). ژن جدید از میانگین چهار ژن همان کروموزوم محاسبه شده و جایگزین ژن اول (a) می‌گردد:

$$\text{NewGene} = \text{round}\left(\frac{a + b + c + d}{۴} \right)$$

۵. **بقای شایسته‌ترین‌ها (Survival of the Fittest):** از میان ۶ کروموزوم حاضر (۴ والد، ۱ فرزند بدون تغییر و ۱ فرزند جهش‌یافته)، ۴ کروموزوم با بیشترین برازش برای نسل بعد برگزیده می‌شوند.

الف) ردیابی دستی ۳ نسل متوالی (Manual Trace): مراحل فوق را برای ۳ نسل متوالی ($t = ۱, ۲, ۳$) به طور کامل اجرا کنید. برای هر نسل، جدول محاسبات برازش، والدین انتخاب‌شده، فرزندان حاصل از تقاطع، جزییات جهش و ۴ کروموزوم برگزیده نهایی را بنویسید.

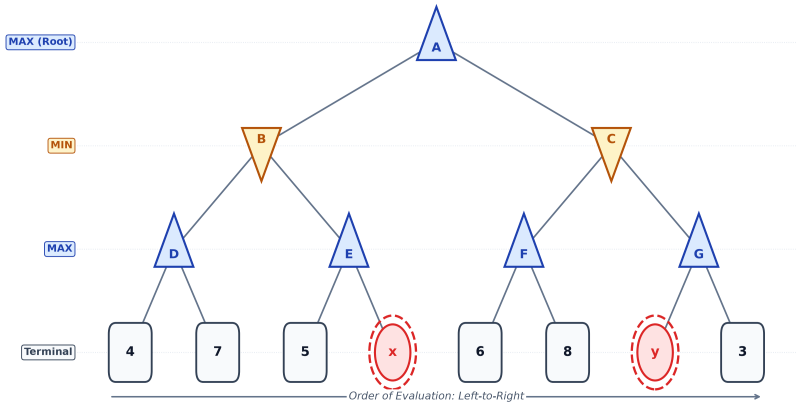
ب) چرا وجود عملگر جهش (Mutation) در الگوریتم ژنتیک حیاتی است؟ در صورت غیرفعال بودن جهش ($p_m = ۰$)، چه محدودیتی در پوشش فضای جستجو ایجاد می‌شود؟

ج) اگر در هر نسل همواره فقط بهترین کروموزوم‌ها برای بقا و بازتولید انتخاب شوند (Over-selection Pressure)، چه پدیده‌ای در تنوع ژنتیکی رخ می‌دهد و چه اثری بر فرآیند بهینه‌سازی دارد؟

د) چرا الگوریتم ژنتیک در مقایسه با الگوریتم‌های گرادیانی و محلی مانند تپه‌نوردی با احتمال کمتری در بهینه‌های محلی به دام می‌افتد؟

سؤال ۵ [۲۰ نمره]

درخت بازی زیر را در نظر بگیرید. گره ریشه (A) یک گره MAX است و سطوح بعدی به‌صورت متناوب بین گره‌های MIN و MAX جابجا می‌شوند. الگوریتم هرس آلفا-بتا (Alpha-Beta Pruning) با فرض پیمایش چپ به راست بر روی درخت اجرا می‌گردد. مقادیر دو برگ x و y مجهول و متغیرهای حقیقی هستند ($x, y \in \mathbf{R}$).



الف) مقدار Minimax هر یک از گره‌های B و C را بر حسب پارامترهای x و y محاسبه کنید و پاسخ را به صورت تابع چندضابطه‌ای بنویسید.

ب) شرایط دقیق ریاضی بر روی x و y را تعیین کنید که تحت آن مقدار نهایی ریشه برابر با مقدار زیردرخت سمت چپ (B) باشد ($\text{val}(A) = \text{val}(B)$).

ج) فرض کنید الگوریتم تا قبل از رسیدن به برگ x اجرا شده است. با تحلیل بازه مقادیر $[\alpha, \beta]$ در گره E، مشخص کنید به ازای چه مقادیری از x، این برگ هرس می‌شود و نیازی به ارزیابی آن نیست؟

د) تحلیل هرس را برای برگ y و گره پس از آن (برگ با مقدار ۳) تکرار کنید. مشخص کنید آیا شرایط هرس در زیردرخت C مشابه زیردرخت B است یا خیر؟

هـ) دو مقدار نمونه برای x و y انتخاب کنید به گونه‌ای که هرس در هر دو سمت درخت رخ دهد (مانند $x = ۲$ و $y = ۹$). سپس مراحل اجرای گام‌به‌گام الگوریتم هرس آلفا-بتا، بازه $[\alpha, \beta]$ در هر گام و شاخه‌های هرس‌شده را مشخص نمایید.

سؤال ۶ [۱۵ نمره]

گره والد p در درخت MCTS تاکنون ۱۲۵ بار بازدید شده است ($N(p) = ۱۲۵$). این گره دارای ۳ فرزند به نام‌های $n_۱$ و $n_۲$ و $n_۳$ است. پاداش تمام شبیه‌سازی‌ها در بازه $[۰, ۱]$ قرار دارد. انتخاب فرزند با فرمول استاندارد UCB1 صورت می‌گیرد:

$$UCB۱(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}, \qquad C = \sqrt{۲}, \quad \ln = \log_e$$

اطلاعات آماری فرزندان در جدول زیر آمده است ($\ln(۱۲۵) \approx ۴٫۸۲۸۳$):

فرزند (Child)	تعداد بازدید ($N(n)$)	مجموع پاداش ($U(n)$)
$n_۱$	۱۰۰	۸۰
$n_۲$	۲۰	۱۸
$n_۳$	۵	۵

الف) محاسبه پایه‌ای: برای هر سه فرزند، مقدار جمله بهره‌برداری (Exploitation)، جمله کاوش (Exploration) و امتیاز نهایی UCB1 را تا ۲ رقم اعشار حساب کنید.

ب) تحلیل انتخاب: کدام فرزند توسط الگوریتم انتخاب می‌شود؟ سهم هر یک از جملات بهره‌برداری و کاوش در امتیاز هر فرزند چقدر است؟ در صورتی که تصمیم‌گیری صرفاً بر اساس بهره‌برداری محض (Pure Exploitation) انجام می‌گرفت، کدام فرزند انتخاب می‌شد و چرا چنین انتخابی قابل اعتماد نیست؟

ج) دینامیک باخت‌های متوالی: فرض کنید فرزند منتخب در قسمت (ب) انتخاب شده و در شبیه‌سازی‌های بعدی پیاپی شکست بخورد ($r = ۰$). در هر گام N این گره و $N(p)$ یک واحد افزایش می‌یابند. حداقل چند باخت متوالی لازم است تا الگوریتم گره دیگری را انتخاب کند؟ تغییرات امتیازها را نشان داده و بیان کنید چرا بررسی وضعیت گره $n_۱$ در این سناریو لزومی ندارد.

د) تحلیل سابقه بی‌نقص و اثر پارامتر کاوش: فرض کنید $N(p) = ۱۲۵$ و آمار $n_۱$ و $n_۲$ ثابت بماند، اما تمام شبیه‌سازی‌های $n_۳$ با برد همراه باشند ($U(n_۳) = N(n_۳)$).

- حداقل مقدار $N(n_۳)$ چقدر باشد تا الگوریتم گره $n_۲$ را به $n_۳$ ترجیح دهد؟

- شرط سبقت را بر حسب پارامتر کاوش دلخواه C به دست آورید. به ازای چه مقادیری از C سبقت گره $n_۲$ ناممکن خواهد بود؟
- رفتار و تصمیم سیستم را در حالت حدی $C \to \infty$ تحلیل کنید.

سؤالات عملی (۱۰۰ نمره)

سؤال ۱ **[۶۰ نمره]**

در این تمرین عملی، شما یک سیستم هوشمند برای حل مسئله‌ی «برنامه‌ریزی و بهینه‌سازی سفر شهری» بر مبنای الگوریتم‌های Local Search طراحی و پیاده‌سازی خواهید کرد.
برخلاف روش‌های جستجوی گام‌به‌گام و کلاسیک مسیر (نظیر A* یا Dijkstra) که مسیر را رأس‌به‌رأس گسترش می‌دهند، در جستجوی محلی مسئله به صورت «پیکربندی کامل» (Complete-State Configuration) مدل‌سازی می‌شود. در دنیای واقعی، زمان سفر تنها تابعی از توپولوژی معابر نیست؛ بلکه شدیداً به «ساعت حرکت» وابسته است (برای مثال حرکت در ساعت 07:15 ترافیک کاملاً متفاوتی نسبت به ساعت 08:00 ایجاد می‌کند).

تعریف وضعیت و تابع هدف:
هر وضعیت در فضای حالت به صورت یک تاپل دوتایی تعریف می‌شود:

$$x = \langle P,\ t_{\text{start}} \rangle$$

- $P = \langle v_0, v_1, \ldots, v_m \rangle$ یک مسیر پیوسته و بدون دور (loop-free) از مبدأ ($v_0 = \text{Home}$) تا مقصد ($v_m = \text{Uni}$) روی گراف جهت‌دار شبکه معابر است.
- t_{start} ساعت حرکت از مبدأ در بازه‌ی زمانی مجاز روزانه است:

$$\mathcal{T} = \{ t_{\min} + k \cdot \Delta t \mid k \in \mathbf{N}_0,\ t_{\min} + k \cdot \Delta t \leq t_{\max} \}$$

زمان عبور از هر یال $e = (u, v)$ وابسته به زمان دقیق ورود خودرو به آن یال است:

$$\tau(e, t) = t_e^{\text{base}} \times \big(1 + C_e(t)\big)$$

که در آن $C_e(t)$ ضریب ترافیک متغیر با زمان بر مبنای امواج گوسی ساعت اوج شلوغی است.
هدف، یافتن وضعیتی است که تابع هزینه سفر $f(x)$ را کمینه کند (محاسبه در شبیه‌ساز FIFO با پیش‌روی زمان). وضعیت‌های دارای دور، گسستگی در مسیر یا ساعت حرکت خارج از بازه، مشمول جریمه بی‌نهایت ($f(x) = \infty$) خواهند شد.

بخش‌های پیاده‌سازی (پوشه /algorithms):
شما باید فایل‌های موجود در پوشه /algorithms را بدون تغییر ساختار امضای توابع پیاده‌سازی نمایید:

۱. **عملگرهای همسایگی و ترکیب (operators.py):**

- تابع get_neighbors: تولید همسایگان معتبر با اعمال جهش روی ساعت حرکت ($\pm\Delta t$) و تغییر موضعی زیرمسیرها (ایجاد detour بدون دور).
- تابع crossover: ترکیب ساختار دو مسیر والد بر مبنای تقاطع‌های مشترک بینابینی و میانگین‌گیری از ساعت حرکت.

۲. **الگوریتم Hill Climbing (hill_climbing.py):** پیاده‌سازی Steepest Descent با پشتیبانی از حرکات افقی روی دشت‌ها (Sideways Moves) تا سقف مشخص و بازنشانی‌های تصادفی (Random Restarts) برای خروج از بهینه‌های محلی.

۳. **الگوریتم Simulated Annealing (simulated_annealing.py):** پیاده‌سازی برنامه زمان‌بندی افت دما ($T_{k+1} = \alpha T_k$) و پذیرش احتمالی وضعیت‌های بدتر طبق معیار Metropolis ($P = \exp(-\Delta E / T)$).

۴. **الگوریتم Local Beam Search (beam_search.py):** نگهداری هم‌زمان K وضعیت موازی، تجمیع همه‌ی همسایگان حاصل از این K وضعیت و انتخاب مجدد K وضعیت برتر از میان کل کاندیداها.

۵. **الگوریتم Genetic Algorithm (genetic_algorithm.py):** مدیریت جمعیت، تعریف تابع Fitness، انتخاب والدین (Tournament یا Roulette Wheel)، بازترکیب با crossover، جهش تصادفی با نرخ معین و حفظ نخبگان (Elitism).

مدیریت ابرپارامترها (config.json):
تمامی پارامترهای تنظیمی الگوریتم‌ها در فایل config.json متمرکز شده‌اند. شما می‌توانید این مقادیر را برای دستیابی به بهترین کارایی تغییر دهید یا متغیرهای جدید دلخواه خود را در آن تعریف کنید.

ابزارهای بصری‌سازی و اجرا:
برای مشاهده رفتار ترافیکی شبکه قبل از شروع کدنویسی، می‌توانید از ابزار تعاملی زیر استفاده نمایید:

```
python main.py --interactive
```

همچنین برای اجرای الگوریتم‌ها، رسم مسیر بهینه و رسم نمودار همگرایی ($f(x)$ بر حسب تکرار)، از آرگومان‌های plot-route– و plot-metrics– در فایل main.py استفاده کنید.

گزارش کار تحلیلی (REPORT.md یا PDF):
همراه با کدها، ارائه‌ی یک گزارش الزامی است. این گزارش می‌تواند به **زبان فارسی یا انگلیسی** نوشته شود (انتخاب با خود دانشجو است).
نکته بسیار مهم: انتظار متن‌های طولانی و کلیشه‌ای نداریم؛ پاسخ‌های دقیق، متفکرانه و مختصر مد نظر است. نگارش گزارش **اکیداً باید توسط خود دانشجو** انجام شود و استفاده از ابزارهای LLM برای تولید متن گزارش ممنوع است.

موارد الزامی در گزارش:

- شرح منطق پیاده‌سازی عملگرهای همسایگی و چگونگی تضمین بدون دور ماندن مسیرها.
- جدول مقایسه‌ای عملکرد هر 4 الگوریتم (بهترین زمان سفر $f(x^*)$، ساعت حرکت بهینه، زمان اجرای برنامه و تعداد گام تا همگرایی).
- قرار دادن تصویر نمودار همگرایی (convergence_comparison.png) و تحلیل مقایسه‌ای رفتار الگوریتم‌ها روی دشت‌ها، قله‌ها و دره‌های محلی.
- **پاسخ به حداقل 5 پرسش از 7 پرسش تحلیلی زیر** (پاسخ‌های شما بررسی شده و بهترین 5 پاسخ نمره‌دهی خواهند شد):

۱. **پرسش اول (Continuous Optimization & Gradient Descent):** در مسائل بهینه‌سازی پیوسته، الگوریتم Gradient Descent متغیرها را با رابطه‌ی $x \leftarrow x - \alpha \nabla f(x)$ به‌روزرسانی می‌کند.

- چرا الگوریتم Gradient Descent کلاسیک اساساً برای بهینه‌سازی مؤلفه مسیر (P) در این گراف معابر قابل استفاده نیست؟
- آیا می‌توان از Gradient Descent صرفاً برای بهینه‌سازی ساعت حرکت t_{start} (با فرض ثابت بودن مسیر P) استفاده کرد؟ تابع ترافیک پویا ($C_e(t)$) چه شرایط ریاضی مشخصی را باید ارضا کند تا چنین کاری مجاز باشد؟

۲. **پرسش دوم (Local Search vs. Classical Incremental Search):** الگوریتم‌های کلاسیک جستجوی گراف (مانند A* یا Dijkstra) مسیر را به صورت گام‌به‌گام می‌سازند.

- در این مسئله، چرا برنامه‌ریزی سفر به صورت یک پیکربندی کامل ($x = \langle P, t_{\text{start}} \rangle$) مدل شده است و نه جستجوی فضای حالت تدریجی؟
- تحت چه شرایط یا محدودیت‌هایی در شبکه‌های شهری، الگوریتم‌های تدریجی مانند Time-Dependent Dijkstra در مقایسه با روش‌های Local Search دچار چالش یا ناکارآمدی می‌شوند؟

۳. **پرسش سوم (Local Beam Search vs. Genetic Algorithm):** در اسلایدهای درس، الگوریتم Local Beam Search به عنوان نگهداری K جستجوی موازی معرفی شده و به شباهت آن با ایده‌ی تکامل زیستی (Evolution) اشاره می‌شود.

- عملگر Crossover در الگوریتم ژنتیک چه مزیت بنیادینی نسبت به مکانیزم انتخاب حریصانه‌ی ادغام‌شده در Local Beam Search ایجاد می‌کند؟
- آیا الگوریتم Local Beam Search شما دچار مشکل کلاسیک همگرا شدن هر K پرتو به یک کمینه محلی تکراری (Population Collapse) شد؟ چگونه می‌توان تنوع پرتوها را حفظ کرد؟

۴. **پرسش چهارم (Plateaus & Sideways Moves):** در مسئله‌ی 8-Queens، دادن مجوز تا سقف 100 حرکت افقی (Sideways Moves)، شانس موفقیت تپه‌نوردی را از 14% به 94% رساند.

- در این شبکه معابر، چه سناریوی واقعی باعث ایجاد دشت یا وضعیت‌های مسطح (Shoulder / Flat Local Extrema) می‌شود که در آن همسایگان زمان سفر یکسان یا تقریباً یکسانی دارند؟
- تنظیم مقدار ابرپارامتر max_sideways چه تأثیری روی زمان اجرای الگوریتم در برابر کیفیت جواب نهایی داشت؟

۵. **پرسش پنجم (Cooling Schedule Dynamics):** الگوریتم Simulated Annealing بر پایه‌ی احتمال پذیرش متروپولیس $P = \exp(-\Delta E/T)$ عمل می‌کند.

- ریسک‌های انتخاب دمای اولیه‌ی (T_0) بسیار بالا یا بسیار پایین چیست؟
- اگر نرخ خنک‌سازی (α) خیلی کوچک باشد (مانند $\alpha = ۰/۵$ یا سردسازی ناگهانی/Quenching) در مقایسه با حالتی که بسیار نزدیک به یک باشد (مانند $\alpha = ۰/۹۹۹$)، چه تغییری در رفتار جستجو مشاهده کردید؟

۶. **پرسش ششم (Crossover Disjointness & Epistasis):** دو جواب والد $x_1 = (P_1, t_1)$ و $x_2 = (P_2, t_2)$ را در نظر بگیرید که از دو مسیر کاملاً مجزا از شهر عبور می‌کنند و هیچ گره مشترکی به جز مبدأ و مقصد ندارند.

- تابع crossover شما چگونه با این حالت برخورد می‌کند تا خروجی نامعتبر یا دارای قطع‌شدگی تولید نشود؟
- چرا ترکیب دو مسیر والد باکیفیت ممکن است در مواردی فرزندی با کیفیت بسیار ضعیف و نامطلوب تولید کند (High Epistasis)؟

۷. **پرسش هفتم (Combinatorial State Space):** فرض کنید گراف دارای $|V|$ گره، میانگین درجه خروجی b برای هر گره، میانگین طول مسیر L گام (یال) از مبدأ تا مقصد، و بازه زمانی حرکت با N_t گسست زمانی باشد.

- یک کران بالای تئوری برای تعداد کل وضعیت‌های فضای حالت ($|\mathcal{X}|$) فرمول‌بندی کنید.
- با توجه به این فرمول، توضیح دهید چرا جستجوی کامل و جامع (Brute-force) برای شبکه‌های واقعی غیرممکن است و استفاده از الگوریتم‌های Local Search ضرورت پیدا می‌کند.

نحوه تحویل: شما باید این مخزن را در حساب گیت‌هاب شخصی خود Fork کرده و با کامیت‌های منظم پیشرفت کار خود را ثبت نمایید. در نهایت لینک مخزن خود را در سامانه تحویل ارسال کنید.
توجه: تمامی تغییرات باید قبل از ددلاین کامیت و پوش شده باشند. هرگونه تغییر یا کامیت پس از ددلاین مشمول جریمه سنگین یا نمره صفر در آن بخش خواهد شد.

لینک مخزن کدهای اولیه برای fork کردن: https://github.com/AliMozdian/commute-optimizer

سؤال ۲ **[۴۰ نمره]**

این بازی مشابه بازی Pacman می باید با این تفاوت که نقطه ای برای امتیاز گرفتن وجود ندارد و تنها راه امتیاز گرفتن در این بازی به قتل رساندن روح هاست.

در هر استیت از بازی ۲ اکشن مجاز وجود دارد: حرکت به یکی از tile های همسایه یا کشتن یکی از روح هایی که در tile های همسایه. کشتن یک روح در tile همسایه موجب رفتن کاراکتر به آن tile می شود.

در ابتدای بازی یک تیر دریافت می کنید و به ازای پیمایش هر ۱۰ خانه یک تیر جدید به شما داده می شود.
محیط بازی هر بار به صورت رندم با تعدادی دیوار تولید می شود.
هدف ما زنده ماندن و کشتن بیشترین تعداد روح هاست.

برای این تمرین الگوریتم های minimax ، alphabeta-pruning و Montecarlo-Treesearch را باید به کمک TODO های موجود در فایل داده شده پیاده سازی کنید.

بازی بعد از ۱۰۰ حرکت به طور اتوماتیک برای جلوگیری از حالت های stale متوقف می شود.

در الگوریتم های Alphabeta و minimax ، عمق جستجو دست خود شماست
در الگوریتم MonteCarlo تعداد iteration ها و rollout ها و همچنین rollout policy دست خود شماست.
زمان پردازش برای هر حرکت نمی تواند بیشتر از ۵ ثانیه باشد و در صورتی که از محدودیت زمانی عبور کنید حرکتی انجام نخواهید داد.

با توجه به اینکه نمی توان در این زمان محدود به کمک minimax و alphabeta تا terminal state پیش رفت ، پیاده سازی یک evaluation-heauristic برای محدود کردن عمق سرچ ضروری است.

همچنین امتیاز دهی به اکشن ها و رویداد های زیر با خودتان است:
- قدم برداشتن
- کشتن روح
- مردن
- دریافت کردن تیر

پارامتر های پیشنهادی برای استفاده در تابع evalution :
- فاصله Manhattan تا روح ها
- متوسط فاصله تا دیوار
- تعداد تیرها
- تعداد tile مانده تا دریافت تیر جدید
- تعداد روح ها

در انتها در داک پیاده سازی خود عملکرد الگوریتم های متفاوت را مقایسه کنید و اگر الگوریتمی عملکرد خوبی ندارد دلیل آن را توضیح دهید.

همچنین می توانید از assess– به عنوان آرگومان ورودی استفاده کنید و سپس یک عدد به عنوان تعداد مشخص کنید تا همه ی الگوریتم ها شبیهسازی شود و متوسط نتایج برای شما چاپ شود.